■ Design Uber / Lyft

System Design Interview | Location-Based Services

algomaster.io | Deep Dive Notes

■ Key Requirements

Functional	Rider requests ride with location; Match to nearby driver; Real-time driver tracking; Price estimation; Trip management; Payments
Non-Functional	Matching latency <1s; Driver location updates every 4s; 99.99% availability; Global scale; Handle surge pricing
Scale	1M concurrent drivers; Location update: 1M × 1 update/4s = 250K writes/sec; geospatial indexing at scale

■ High-Level Architecture

• Location Service: ingests driver GPS updates via WebSocket/HTTP; stores in Redis geospatial index + time-series DB

• Matching Service: on ride request, query nearby drivers via geospatial search → rank by ETA + rating → broadcast to top 3-5 drivers

• Trip Service: manages trip state machine (requested→accepted→in-progress→completed); persists to PostgreSQL

• Pricing Service: dynamic surge pricing based on supply/demand ratio in geo-grid; pre-compute surge factors

■ Geospatial Indexing Deep Dive

• **Geohash:** Encode lat/lng as alphanumeric string; each char ≈ 1/32 precision level; nearby locations share prefix; Redis GEORADIUS uses geohash internally

• **Quadtree:** Recursively divide map into 4 quadrants; leaf nodes contain drivers; dynamic resolution based on density; good for uneven distributions

• **S2 Cells (Google/Uber):** Sphere mapped to cube faces → hierarchical cells at 30 levels; no distortion at poles; Uber uses S2 for geo-fencing and driver assignment

• **Redis GEO:** GEOADD stores lat/lng; GEORADIUS finds drivers within N km; internally uses geohash sorted set; O(N+log M) complexity

■ Driver Matching Algorithm

• Step 1: GEORADIUS query → get all drivers within 5km radius; filter by status=available

• Step 2: Calculate ETA for top candidates using road network (pre-computed or Dijkstra on graph)

• Step 3: Rank by: ETA (primary) + driver rating + acceptance rate + surge zone balance

• Step 4: Broadcast to top 3-5 drivers simultaneously; first to accept wins; timeout → next batch

• Optimization: driver clustering — pre-assign drivers to grid cells; matching service only queries relevant cells

■ Location Storage Strategy

• Real-time location: Redis (in-memory, fast, geospatial native); TTL 30s — stale if driver disconnects
• Historical location: Apache Kafka → S3 / ClickHouse for analytics and replay; time-series format
• Trip route: PostgreSQL (PostGIS extension) for precise route storage and geo-queries in trip data
• Driver supply heatmap: pre-aggregate per S2 cell every 1min → cache in Redis for surge price computation

■■ Communication Protocol

WebSocket (Driver App)	HTTP Polling (Rider App)
✓ Persistent connection	✓ Simpler implementation
✓ Real-time location push	✓ Poll every 3-5s for ETA
✓ Lower battery drain	✓ Acceptable latency for rider
✓ Complex reconnect handling	✓ Higher request overhead

■ Surge Pricing System

• Divide city into hexagonal grid cells (H3 library by Uber); compute supply/demand ratio per cell
• Surge multiplier = f(demand/supply) — piecewise linear or ML model; price updated every 1-5 minutes
• Rider sees surge before confirming; price lock for 2 mins after quote to prevent bait-and-switch
• Anti-gaming: surge price visible on map; encourages drivers to move to high-surge areas voluntarily